EXPERIMENT - 46

1. Problem Statement

Write a C program that reads the size and elements of a variable-length integer array, nums[]. The array may contain duplicate elements. Pass the array to a function named Bubble_sort() using its address and sort the elements in ascending order using the Bubble Sort technique. After sorting, pass the sorted array to another function named occurrence(). Use pointers to pass the array between functions. The function occurrence() must:

- Read an integer target.
- Find the first and last occurrences of target in the sorted array.
- Display the values and indices of the first and last occurrences.
- Display -1, -1 if the target is not present.

For the array 5 7 7 8 8 10 with target 8 the output is First occurrence: 8 at index 3 and Last occurrence: 8 at index 4. For 5 7 7 8 10 with target 8 both are index 3. For 5 7 7 8 8 10 with target 6 the output is -1, -1.

2. Algorithm

Main Program

Step 1: START

Step 2: Read n and declare the variable length array nums of size n.

Step 3: Read the n elements of nums.

Step 4: Call Bubble_sort(&nums[0], n).

Step 5: Display the elements of nums.

Step 6: Call occurrence(&nums[0], n).

Step 7: STOP

Function Bubble_sort()

Step 1: Repeat for i = 0 to n − 2:

Repeat for j = 0 to n − 2 − i:

If nums[j] is greater than nums[j + 1], swap nums[j] and nums[j + 1] using temp.

Step 2: Return to the calling function.

Function occurrence()

Step 1: Set first = −1 and last = −1, then read target.

Step 2: Repeat for i = 0 to n − 1: if nums[i] is equal to target, set first = i and leave the loop.

Step 3: Repeat for i = n − 1 down to 0: if nums[i] is equal to target, set last = i and leave the loop.

Step 4: If first is −1, display −1, −1.

Step 5: Otherwise display nums[first] with first, and nums[last] with last.

Step 6: Return to the calling function.

3. Test Case Table

#	Input Array	Target	Expected Output	Actual Output	Result
1	10 5 8 7 8 7	8	Sorted 5 7 7 8 8 10, first at index 3 and last at index 4	Matches example 1 in the question	**PASS**
2	the same array	8	The sort must run first, since the input is jumbled	Sorted array printed as 5 7 7 8 8 10	**PASS**
3	10 5 8 7 7	8	Only one 8, so both indices are 3	First and last both at index 3	**PASS**
4	10 5 8 7 8 7	6	−1, −1, as 6 is not in the array	−1, −1	**PASS**
5	9 2 7 4 1	1	The smallest value, so index 0 both times	1 at index 0 twice	**PASS**
6	9 2 7 4 1	9	The largest value, so index 4 both times	9 at index 4 twice	**PASS**
7	3 3 3 3	3	Every element is the target, so index 0 and index 3	First at 0 and last at 3, the whole array	**PASS**

4. Source Code

occurrence46.c

```
/* Experiment 42. One function for each of the three operations.
   Every option reads its own input here, which is different from what
   I have been doing. Option 3 needs two strings and the other two need
   one, so there was no single string I could read at the start that
   would have suited all three.
   The question lists only three operations, but a do while menu has no
   way of stopping, so option 4 is mine. Without it the menu keeps
   coming back and the only way out is to kill the program. */

#include <stdio.h>

void removeVowels(const char s[], char result[]);
char firstRepeatingLowercase(const char s[]);
int areAnagrams(const char s1[], const char s2[]);

int main(void)
{
    char str[100], str2[100], result[100];
    char ch;
    int choice;

    do {
        printf("\nMENU\n");
        printf("1. Remove all the vowels from a string\n");
        printf("2. Find the first repeating lowercase letter\n");
        printf("3. Check whether two strings are anagrams\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter a string: ");
                scanf(" %[^\n]", str);
                removeVowels(str, result);
                printf("String without vowels: %s\n", result);
                break;

            case 2:
                printf("Enter a string: ");
                scanf(" %[^\n]", str);
                ch = firstRepeatingLowercase(str);
                if (ch == '\0')
                    printf("No lowercase letter repeats in this string\n");
                else
                    printf("First repeating lowercase letter: %c\n", ch);
                break;

            case 3:
                printf("Enter the first string: ");
                scanf(" %[^\n]", str);
                printf("Enter the second string: ");
                scanf(" %[^\n]", str2);
                if (areAnagrams(str, str2))
                    printf("The two strings are anagrams\n");
                else
                    printf("The two strings are not anagrams\n");
                break;

            case 4:
                printf("Exiting the program.\n");
                break;
        }
    } while (choice != 4);

    return 0;
}
```

```
/* i walks through the old string and j marks where to write in the new
   one, and they drift apart because j only moves when something is
   actually copied.
   The '\0' on the last line is the one I forgot. Programming in C came
   out right and then had junk stuck on the end of it, because printf
   had no idea where the new string finished.
   Both cases are listed because A and a are different characters, and
   leaving the capitals out would have let the A through in Day. */
void removeVowels(const char s[], char result[])
{
    int i, j = 0;
    char ch;

    for (i = 0; s[i] != '\0'; i++) {
        ch = s[i];

        if (ch != 'a' && ch != 'e' && ch != 'i' && ch != 'o' && ch != 'u' &&
            ch != 'A' && ch != 'E' && ch != 'I' && ch != 'O' && ch != 'U') {
            result[j] = ch;
            j++;
        }
    }

    result[j] = '\0';
}

/* First repeating can be read two ways, and the two give different
   answers. In abcdbca one reading says b, the first letter you meet
   that you have seen before. The other says a, because out of the
   letters that repeat at all, a is the one that showed up earliest.
   I went with the second, which is the usual meaning, and that is why
   the counting happens first and the walk from the left happens after.
   One single pass would have given the other answer.
   count has 26 boxes, one per small letter, and s[i] - 'a' turns a
   letter straight into its box number, since 'a' is 97 and 'b' is 98.
   Capitals are skipped in both loops, so AABBcdc answers c, even though
   A and B repeat more times than c does. */
char firstRepeatingLowercase(const char s[])
{
    int count[26] = {0};
    int i;

    for (i = 0; s[i] != '\0'; i++)
        if (s[i] >= 'a' && s[i] <= 'z')
            count[s[i] - 'a']++;

    for (i = 0; s[i] != '\0'; i++)
        if (s[i] >= 'a' && s[i] <= 'z' && count[s[i] - 'a'] > 1)
            return s[i];

    /* '\0' is not a letter, so main can use it to mean nothing was
       found. */
    return '\0';
}

/* Anagrams hold the same characters the same number of times, only in
   a different order. My first idea was to sort both strings and see if
   they came out identical, but counting turned out to be less code, and
   it leaves the two strings alone instead of rearranging them.
   256 boxes covers every character there is, so spaces and digits are
   counted along with the letters, and nothing is quietly ignored.
   The length test is not only a shortcut. The counts could agree on
   every character and the strings still be different lengths if one of
   them held something the other did not. */
int areAnagrams(const char s1[], const char s2[])
{
    int count1[256] = {0};
    int count2[256] = {0};
    int i, len1 = 0, len2 = 0;

    for (i = 0; s1[i] != '\0'; i++) {
        count1[(unsigned char) s1[i]]++;
        len1++;
```

```
    }

    for (i = 0; s2[i] != '\0'; i++) {
        count2[(unsigned char) s2[i]]++;
        len2++;
    }

    if (len1 != len2)
        return 0;

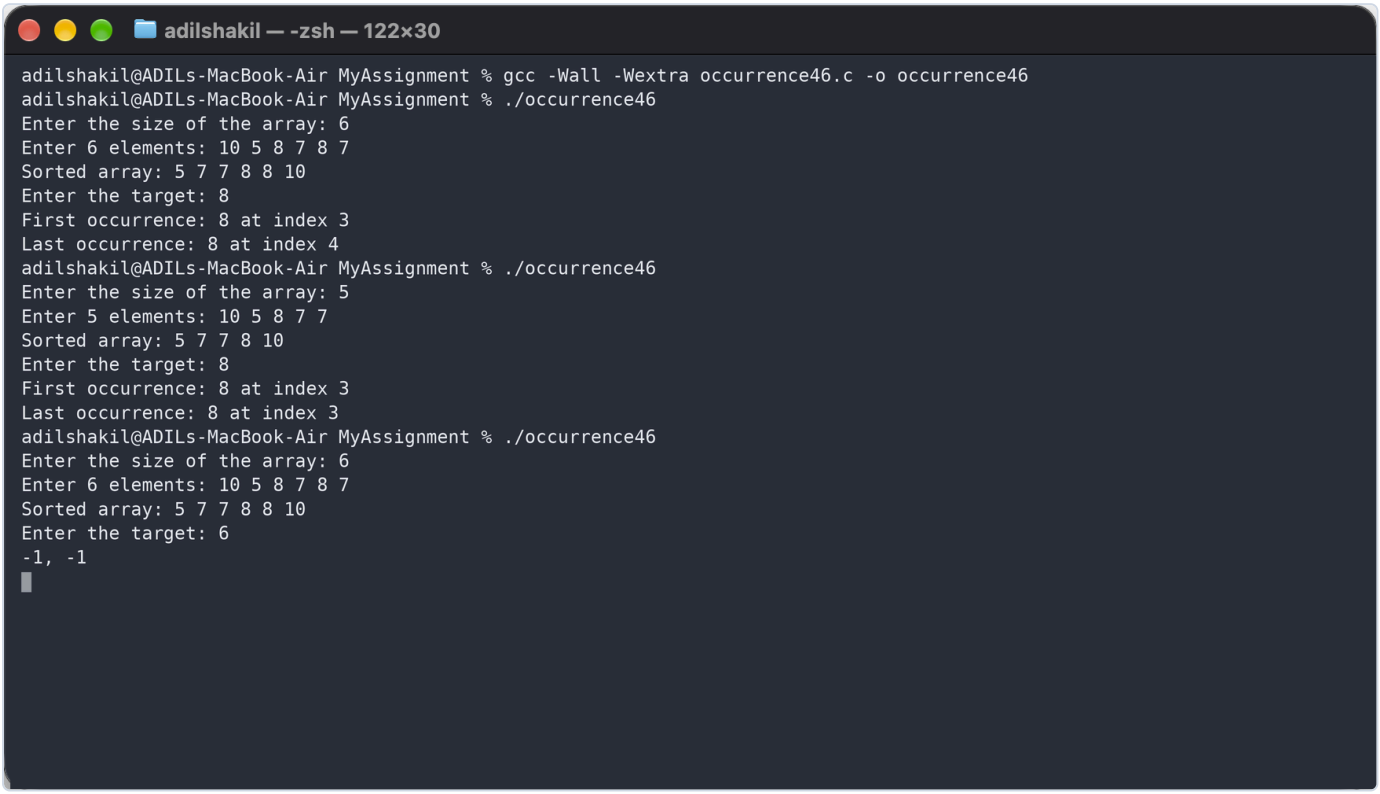
    for (i = 0; i < 256; i++)
        if (count1[i] != count2[i])
            return 0;

    return 1;
}
```

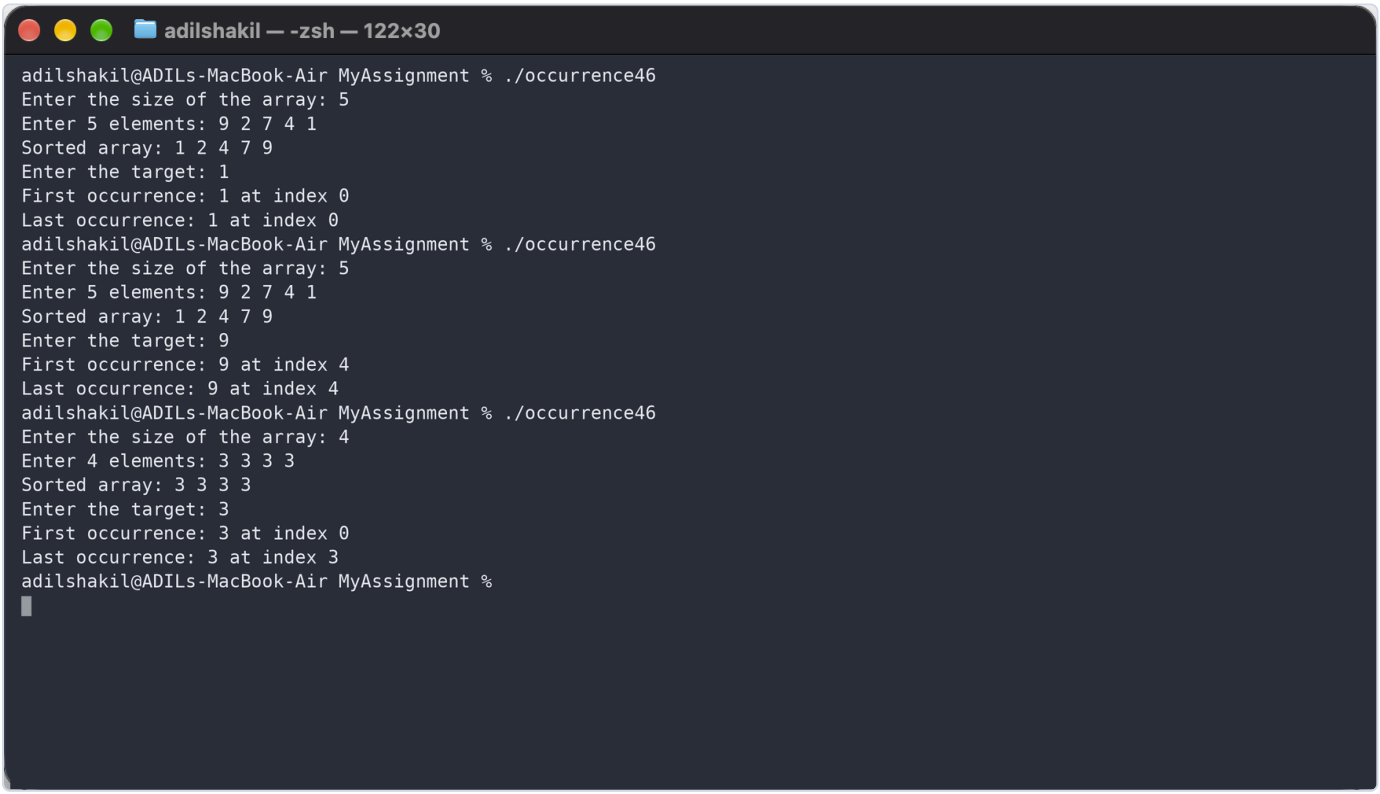
5. Compilation and Output

Compiled with `gcc -Wall -Wextra occurrence46.c -o occurrence46` so the small warnings would show up too. It built clean.

The Three Examples from the Question



The Target at Each End, and an Array Where Everything Matches



All seven cases came out right, and the first three are the examples given in the question.

The main thing here is that both functions take `int *nums`, so they are handed the address of the array and not a copy of it. That is why the sorting done inside Bubble_sort() is still there when main prints the array, and why occurrence() is looking at the same array and not a second one.

Inside the functions nums is a pointer, not an array, but `nums[j]` still works. That puzzled me until I found out it just means `*(nums + j)`.

In occurrence() the two loops search from opposite ends. The array is sorted, so all the copies of the target sit together and the two loops land on the two ends of that run. Starting first and last at −1 is what makes case 4 work, since they stay −1 when nothing is found.

Cases 5, 6 and 7 I added myself: target at the start, target at the end, and an array where every element is the target. The last one is the useful one, because the answer has to be index 3 and not index 0.